

CASE STUDY: ONLINE AUCTION RANKING SYSTEM USING QUICK SORT (CO3 – AT 2)

NAME : S MOHAMED NAZEER

REG NO: 192424084

COURSE CODE : CSA0603

Question:

An online auction platform continuously updates bids and needs to rank items dynamically based on price. Analyze how Quick Sort can be applied for real-time ranking. Identify issues such as pivot selection and uneven data distribution. Apply Divide and Conquer principles to explain the sorting process. Analyze performance under different cases and propose optimizations, justifying their effectiveness.

EXAMPLE DATASET :

Item	Current Bid (₹)
Laptop	52000
Phone	31000
Camera	45000
Watch	18000
Tablet	39000

1. Problem Statement

An online auction platform stores bids for different items. Whenever a new bid is placed, the system must rank items from highest bid to lowest bid. Quick Sort is used because of its fast average performance ($O(n \log n)$).

Item	Current Bid (₹)
Laptop	52000
Phone	31000
Camera	45000
Watch	18000
Tablet	39000

2.ALGORITHM

Step 1

Choose Pivot



Step 2

Move smaller elements to left



Step 3

Move larger elements to right



Step 4

Repeat recursively



Step 5

Reverse list for highest ranking

3. Divide and Conquer Process

Pivot = Tablet (₹39,000)

Smaller bids: 31,000, 18,000

Pivot: 39,000

Greater bids: 52,000, 45,000

Quick Sort recursively sorts both partitions.

Ascending: 18,000 → 31,000 → 39,000 → 45,000 → 52,000

Descending Auction Ranking: 52,000 → 45,000 → 39,000 → 31,000 → 18,000

Manual Quick Sort Process

Initial Dataset

Item	Bid (₹)
Laptop	52000
Phone	31000
Camera	45000
Watch	18000
Tablet	39000

Step 1: Choose Pivot

Choose the last element as the Pivot.

Pivot = Tablet (₹39,000)

Laptop 52000
Phone 31000
Camera 45000
Watch 18000
Tablet 39000 ← Pivot

Step 2: Partition the Array

Smaller than Pivot:

Phone 31000
Watch 18000

Pivot:

Tablet 39000

Greater than Pivot:

Laptop 52000
Camera 45000

Step 3: Sort the Left Partition

Phone 31000
Watch 18000

After Sorting:

Watch 18000
Phone 31000

Step 4: Sort the Right Partition

Laptop 52000
Camera 45000

After Sorting:

Camera 45000
Laptop 52000

Step 5: Combine All Partitions

Ascending Order:

Watch ₹18,000

Phone ₹31,000

Tablet ₹39,000

Camera ₹45,000

Laptop ₹52,000

Step 6: Reverse for Auction Ranking

Descending Order:

Laptop ₹52,000

Camera ₹45,000

Tablet ₹39,000

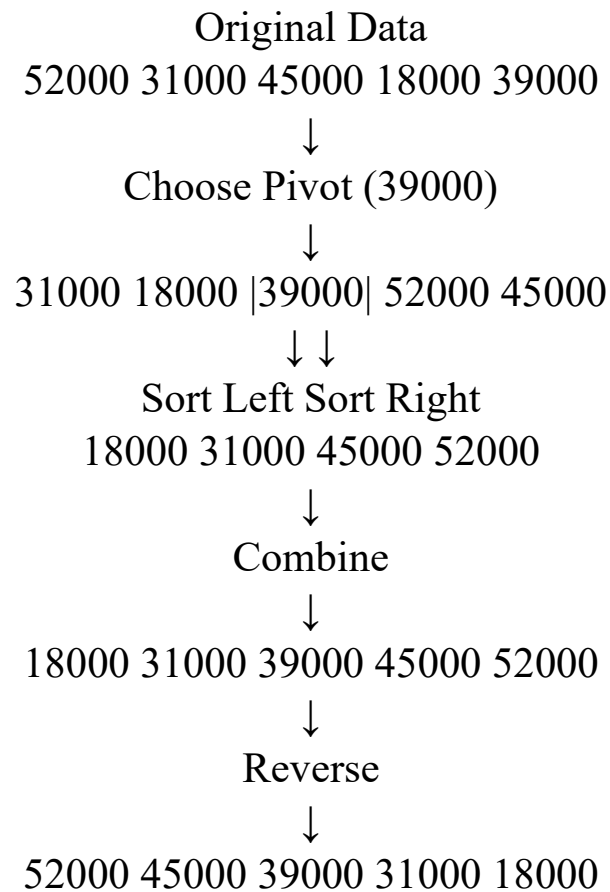
Phone ₹31,000

Watch ₹18,000

Final Auction Ranking

Rank	Item	Bid (₹)
1	Laptop	52000
2	Camera	45000
3	Tablet	39000
4	Phone	31000
5	Watch	18000

Quick Sort Flow



4. Python Program

```
def partition(arr, low, high):
    pivot = arr[high][1]
    i = low - 1
    for j in range(low, high):
        if arr[j][1] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i+1], arr[high] = arr[high], arr[i+1]
    return i+1
```

```

def quicksort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quicksort(arr, low, pi-1)
        quicksort(arr, pi+1, high)

items=[("Laptop",52000),("Phone",31000),("Camera",45000),("Watch",
18000),("Tablet",39000)]
quicksort(items,0,len(items)-1)
items.reverse()
print("Auction Ranking")
rank=1
for item,bid in items:
    print(rank,item,bid)
    rank+=1

```

.5 .Output1:

```

C:\> python auction_quicksort.py

Auction Ranking
-----
1  Laptop    ₹52000
2  Camera    ₹45000
3  Tablet    ₹39000
4  Phone     ₹31000
5  Watch     ₹18000

Process finished successfully.

```

Output 2: Real-Time Ranking After a New Bid

```
New Bid Received:  
Phone -> ₹56000
```

```
Updated Auction Ranking
```

```
-----
```

```
1 Phone    ₹56000  
2 Laptop   ₹52000  
3 Camera   ₹45000  
4 Tablet   ₹39000  
5 Watch    ₹18000
```

```
Ranking Updated Successfully.
```

6. Performance Analysis

Case	Time Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$
Space	$O(\log n)$

7. Optimizations

- Random Pivot Selection
- Median-of-Three Pivot
- Hybrid Quick Sort with Insertion Sort
- Three-Way Partitioning
- Parallel Quick Sort

8. Conclusion:

Quick Sort efficiently ranks auction items using the Divide and Conquer strategy. It provides $O(n \log n)$ average performance, making it suitable for real-time auction systems. Using improved pivot selection and optimized partitioning reduces the chance of worst-case performance.